

ESP32 LAB EXAM

STUDY GUIDE

Adafruit NeoPixel · FastLED · WebSocket

BRING TO EXAM:

✓ ESP32 DevKit ✓ WS2812B LED Strip ✓ Power Supply / Lamp ✓ This Guide
[OPEN REFERENCE] [NO AI] [PER GROUP] [PRACTICAL]

1. Hardware Setup & Wiring

 Power: 30 LEDs at full white = ~1.8A. NEVER power a full strip from ESP32's 3.3V/5V pin. Use external 5V PSU. CRITICAL: Share GND between ESP32 and PSU!

1.1 LED Strip Connections

Strip Wire	Connects To	Notes
5V / +	External 5V PSU	Do NOT use ESP32 onboard 5V for full strip
GND / -	PSU GND + ESP32 GND	MUST share common ground — most common mistake!
Data / DI	GPIO 5 (recommended)	3.3V signal works for most WS2812B; add 300Ω series resistor

 Add a 300–500Ω resistor in series on the data line to protect against signal reflection. Add a 1000μF capacitor across 5V/GND near the strip input to absorb voltage spikes.

1.2 Safe GPIO Pins on ESP32

GPIO	ADC?	Boot Safe?	Recommended Use
GPIO 4	No	Yes	LED data line — best choice
GPIO 5	No	Yes	LED data line — best choice
GPIO 13	Yes	Yes	General I/O
GPIO 14	Yes	Yes	General I/O
GPIO 16	No	Yes	General I/O
GPIO 27	No	Yes	General I/O, buttons
GPIO 34-39	Yes (input only)	Yes	ADC only, NO internal pullup, NO output
GPIO 0	No	NO	Affects boot mode — avoid
GPIO 12	Yes	NO	Affects flash voltage — avoid
GPIO 6-11	No	NO	Connected to flash — NEVER USE

1.3 LED Strip Types

Type	Protocol	Voltage	FastLED Type	NeoPixel Type
WS2812B	1-wire	5V	WS2812B	NEO_GRB + NEO_KHZ800
WS2811	1-wire	5V	WS2811	NEO_RGB + NEO_KHZ400
SK6812	1-wire	5V	SK6812	NEO_GRBW (has White ch.)
APA102	SPI 2-wire	5V	APA102	N/A (SPI not NeoPixel)

2. Adafruit NeoPixel Library

i Install: Arduino Library Manager → search "Adafruit NeoPixel" by Adafruit. Also install "Adafruit BusIO" if prompted.

2.1 Required Include & Object Declaration

```
#include <Adafruit_NeoPixel.h>

#define LED_PIN    5          // GPIO pin connected to strip data line
#define LED_COUNT  30        // Number of LEDs in your strip

// WS2812B: NEO_GRB + NEO_KHZ800 (most common)
// SK6812:  NEO_GRBW + NEO_KHZ800
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
```

2.2 setup() — Always Call These

```
void setup() {
  strip.begin();           // Initialize the strip object
  strip.show();            // Push blank data — turns all LEDs OFF
  strip.setBrightness(50); // 0-255; keep LOW during testing to protect hardware!
}
```

💡 `setBrightness()` is a lossy operation — it permanently scales stored color values. Set brightness ONCE at startup, not dynamically in loop. Use a multiplier in your own code if you need dynamic dimming.

2.3 Core Methods — You Must Know All Of These

Method	What It Does
<code>strip.setPixelColor(i, Color(r,g,b))</code>	Set pixel <i>i</i> to RGB color. Does NOT update strip yet.
<code>strip.show()</code>	PUSH all pending color data to strip. Call after every change.
<code>strip.fill(Color(r,g,b))</code>	Set ALL pixels to one color. Still need <code>show()</code> .
<code>strip.fill(Color(r,g,b), first, count)</code>	Set a RANGE of pixels (first index, count of pixels).
<code>strip.clear()</code>	Set all pixels to OFF (black). Still need <code>show()</code> .
<code>strip.setBrightness(0-255)</code>	Set global brightness (call once in setup).
<code>strip.numPixels()</code>	Returns the number of pixels (LED_COUNT).

Method	What It Does
<code>strip.Color(r, g, b)</code>	Create a 32-bit packed color value from RGB (0-255 each).
<code>strip.Color(r, g, b, w)</code>	RGBW version for SK6812 strips with white channel.
<code>strip.getPixelColor(i)</code>	Get current <code>uint32_t</code> color of pixel <code>i</code> .

⚠ ALWAYS call `strip.show()` after `setPixelColor()` or `fill()` — nothing physically changes on the strip until `show()` is called!

2.4 Color Wipe Pattern

```
void colorWipe(uint32_t color, int waitMs) {
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, color);
    strip.show();
    delay(waitMs);
  }
}

// Call with:
// colorWipe(strip.Color(255, 0, 0), 50); // Red wipe
```

2.5 Theater Chase

```
void theaterChase(uint32_t color, int wait) {
  for (int a = 0; a < 30; a++) { // Repeat 30 times
    for (int b = 0; b < 3; b++) { // 3 offset phases
      strip.clear();
      for (int c = b; c < strip.numPixels(); c += 3)
        strip.setPixelColor(c, color); // Every 3rd LED
      strip.show();
      delay(wait);
    }
  }
}
```

2.6 Rainbow Cycle

```
void rainbowCycle(int wait) {
  for (long firstPixelHue = 0; firstPixelHue < 5 * 65536; firstPixelHue += 256) {
    for (int i = 0; i < strip.numPixels(); i++) {
      int pixelHue = firstPixelHue + (i * 65536L / strip.numPixels());
      strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(pixelHue)));
    }
    strip.show();
    delay(wait);
  }
}

// strip.ColorHSV(hue, sat, val) — hue: 0-65535, sat/val: 0-255
// strip.gamma32(color) — apply gamma correction for natural-looking colors
```

3. FastLED Library

i Install: Arduino Library Manager → search "FastLED" by Daniel Garcia.

3.1 Include & Global Setup

```
#include <FastLED.h>

#define LED_PIN      5
#define NUM_LEDS     30
#define BRIGHTNESS   50
#define LED_TYPE      WS2812B
#define COLOR_ORDER   GRB          // GRB for WS2812B; RGB, BGR, etc. for others

CRGB leds[NUM_LEDS];              // The LED array — THIS is what you modify

void setup() {
  FastLED.addLeds<LED_TYPE, LED_PIN, COLOR_ORDER>(leds, NUM_LEDS);
  FastLED.setBrightness(BRIGHTNESS);
  FastLED.clear();                // Zero out the array
  FastLED.show();                 // Push to strip (all off)
}
```

3.2 CRGB Color — Setting Individual LEDs

```
// Named colors
leds[0] = CRGB::Red;
leds[1] = CRGB::Green;
leds[2] = CRGB::Blue;
leds[3] = CRGB::White;
leds[4] = CRGB::Black;           // = off

// RGB constructor (r, g, b) — values 0-255
leds[5] = CRGB(255, 128, 0);    // Orange

// Hex
leds[6] = CRGB(0xFF8000);       // Orange via hex

// Fade / scale brightness of one pixel
leds[0].fadeToBlackBy(128);     // Dim by 50%

// After ALL changes are done:
FastLED.show();
```

3.3 CHSV — HSV Colors (Great for Effects!)

```
// CHSV(hue, saturation, value)
// hue: 0-255 (wraps around color wheel)
// saturation: 0 (white) to 255 (full color)
// value: 0 (black/off) to 255 (full brightness)

leds[0] = CHSV(0, 255, 255);    // Red
leds[1] = CHSV(85, 255, 255);   // Green
leds[2] = CHSV(170, 255, 255);  // Blue
leds[3] = CHSV(43, 255, 255);   // Yellow

// Cycling rainbow — increment hue each frame
static uint8_t hue = 0;
for (int i = 0; i < NUM_LEDS; i++) {
  leds[i] = CHSV(hue + (i * 10), 255, 255);
}
hue++; // wraps automatically at 255
FastLED.show();
```

3.4 Built-In Fill Functions

```
// Fill ALL pixels with one solid color
fill_solid(leds, NUM_LEDS, CRGB::Blue);
FastLED.show();

// Fill with rainbow spectrum
// fill_rainbow(array, count, startHue, hueDelta)
fill_rainbow(leds, NUM_LEDS, 0, 7); // 7 = hue step between LEDs
FastLED.show();

// Fill with gradient between two colors
fill_gradient_RGB(leds, NUM_LEDS, CRGB::Red, CRGB::Blue);
FastLED.show();

// Fill a RANGE (first, count, color)
fill_solid(leds + 5, 10, CRGB::Green); // LEDs 5-14 only
FastLED.show();
```

3.5 FastLED Core Methods

Method	What It Does
FastLED.show()	Push leds[] array to physical strip. Must call after any change.
FastLED.clear()	Set ALL leds[] to CRGB::Black. Still need show().
FastLED.setBrightness(n)	Global brightness 0-255. Call in setup or between show() calls.
FastLED.delay(ms)	delay() that also refreshes the strip (useful in animations).
FastLED.countLeds()	Returns NUM_LEDS.
leds[i].fadeToBlackBy(n)	Fade pixel toward black by amount n (0=none, 255=full black).
leds[i].nscale8(n)	Scale pixel brightness by n/256.
fill_solid(leds,N,color)	Fill N pixels starting at leds with color.
fill_rainbow(leds,N,h,d)	Fill N pixels with rainbow starting at hue h, step d.
fill_gradient_RGB(...)	Fill with smooth gradient between colors.

3.6 Breathing / Pulse Effect

```
void breathe(CRGB color) {
  // Fade in
  for (int b = 0; b <= 255; b += 3) {
    FastLED.setBrightness(b);
    fill_solid(leds, NUM_LEDS, color);
    FastLED.show();
    delay(10);
  }
  // Fade out
  for (int b = 255; b >= 0; b -= 3) {
    FastLED.setBrightness(b);
    FastLED.show();
    delay(10);
  }
}
```

3.7 Running Rainbow (Non-Blocking with millis)

```
static uint8_t hue = 0;
static unsigned long lastUpdate = 0;

void loop() {
  websocket.loop(); // always first!

  if (millis() - lastUpdate > 20) {
    fill_rainbow(leds, NUM_LEDS, hue++, 7);
    FastLED.show();
    lastUpdate = millis();
  }
}

// NON-BLOCKING: uses millis() instead of delay()
// This lets websocket.loop() keep running!
```

⚠ Never use `delay()` in `loop()` when also using `WebSocket` — it blocks `websocket.loop()` and causes disconnections. Use `millis()`-based timing instead.

4. WebSocket Client

i Required libraries: "WebSocketsClient" by Markus Sattler + "ArduinoJson" by Benoit Blanchon. Install both from Library Manager.

4.1 WiFi Connection

```
#include <WiFi.h>

const char* ssid      = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

void connectWiFi() {
  WiFi.begin(ssid, password);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 30) {
    delay(500);
    Serial.print(".");
    attempts++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nConnected! IP: " + WiFi.localIP().toString());
  } else {
    Serial.println("\nFailed! Restarting...");
    ESP.restart(); // Reboot if can't connect
  }
}
```

4.2 WebSocket Setup & Event Handler

```
#include <WebSocketsClient.h>

WebSocketsClient websocket;
bool isConnected = false;

// Event handler — called automatically by websocket.loop()
void websocketEvent(WStype_t type, uint8_t* payload, size_t length) {
  switch (type) {
    case WStype_CONNECTED:
```

```

    isConnected = true;
    Serial.println("[WS] Connected!");
    sendJoinMessage(); // Send first message right after connecting
    break;

case WStype_DISCONNECTED:
    isConnected = false;
    Serial.println("[WS] Disconnected!");
    break;

case WStype_TEXT:
    handleMessage((char*)payload); // Cast payload to char*
    break;

case WStype_ERROR:
    Serial.println("[WS] Error!");
    break;

case WStype_PING:
case WStype_PONG:
    break; // Library handles keep-alive automatically
}
}

```

4.3 Connecting to the Server

```

const char* SERVER_HOST = "192.168.1.100"; // Server IP address
const int   SERVER_PORT = 8080;           // Port number
// For wss:// (TLS): use beginSSL() instead of begin()

void setup() {
    // ... WiFi connect first ...

    websocket.begin(SERVER_HOST, SERVER_PORT, "/"); // "/" = path
    websocket.onEvent(webSocketEvent);
    websocket.setReconnectInterval(5000); // Auto-reconnect every 5s if dropped
}

void loop() {
    websocket.loop(); // CRITICAL: must be called EVERY loop iteration!
    // ... rest of your code ...
}

```

 **websocket.loop() MUST be called in every loop() iteration. Missing it even once can cause the connection to drop or messages to be missed.**

4.4 Sending JSON Messages

```

#include <ArduinoJson.h>

void sendJoinMessage() {
    StaticJsonDocument<256> doc; // Buffer size in bytes – make it bigger if needed
    doc["type"]   = "join";
    doc["name"]   = "MyDevice";
    doc["color"]  = "blue";

    String message;
    serializeJson(doc, message); // Convert doc to JSON string
}

```

```

    if (isConnected) {
        websocket.sendTXT(message); // Send the string
    }
}

// Sending with nested objects:
void sendStatus(bool ledOn, int brightness) {
    StaticJsonDocument<128> doc;
    doc["type"] = "status";
    JsonObject data = doc.createNestedObject("data");
    data["ledOn"] = ledOn;
    data["brightness"] = brightness;
    String msg; serializeJson(doc, msg);
    websocket.sendTXT(msg);
}

```

4.5 Receiving & Parsing JSON Messages

```

void handleMessage(char* payload) {
    StaticJsonDocument<512> doc; // Must be large enough for incoming message
    DeserializationError error = deserializeJson(doc, payload);

    if (error) {
        Serial.print("JSON parse error: ");
        Serial.println(error.c_str());
        return;
    }

    // Get the "type" field
    const char* type = doc["type"]; // const char* for strcmp

    // IMPORTANT: Use strcmp() to compare const char* strings!
    if (strcmp(type, "color") == 0) {
        int r = doc["r"];
        int g = doc["g"];
        int b = doc["b"];
        // → apply to LED strip
    }
    else if (strcmp(type, "effect") == 0) {
        const char* effect = doc["effect"]; // "rainbow", "wipe", "off"
        // → trigger effect
    }
    else if (strcmp(type, "brightness") == 0) {
        int val = doc["value"];
        strip.setBrightness(val);
        strip.show();
    }
}

```

⚠ NEVER use == to compare const char* strings in C/C++. It compares pointers, not content. Always use strcmp(str1, str2) == 0 to check equality.

4.6 ArduinoJson — Common Operations

Operation	Code
Create doc	StaticJsonDocument<N> doc; (N = bytes; 256 for simple, 512+ for complex)
Set field	doc["key"] = value;

Operation	Code
Get int field	<code>int x = doc["key"];</code>
Get string field	<code>const char* s = doc["key"];</code>
Get bool field	<code>bool b = doc["key"];</code>
Nested object (write)	<code>JsonObject o = doc.createNestedObject("sub"); o["x"] = 1;</code>
Nested object (read)	<code>JsonObject o = doc["sub"]; int x = o["x"];</code>
Array (write)	<code>JsonArray a = doc.createNestedArray("items"); a.add(1);</code>
Serialize to String	<code>String s; serializeJson(doc, s);</code>
Deserialize from char*	<code>DeserializationError e = deserializeJson(doc, payload);</code>
Check parse error	<code>if (e) { Serial.println(e.c_str()); return; }</code>

5. Complete Combined Template — WebSocket + NeoPixel

💡 This is the most likely exam structure. Memorize the skeleton: includes → globals → handler → setup → loop.

```
#include <WiFi.h>
#include <WebSocketsClient.h>
#include <ArduinoJson.h>
#include <Adafruit_NeoPixel.h>

// — Config —————
#define LED_PIN    5
#define LED_COUNT  30
const char* ssid   = "YOUR_WIFI";
const char* password = "YOUR_PASS";
const char* SERVER_HOST = "192.168.1.100";
const int  SERVER_PORT = 8080;

// — Globals —————
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
WebSocketsClient websocket;
bool isConnected = false;

// — Message Handler —————
void handleMessage(char* payload) {
    StaticJsonDocument<512> doc;
    if (deserializeJson(doc, payload)) return; // skip on parse error

    const char* type = doc["type"];

    if (strcmp(type, "color") == 0) {
        strip.fill(strip.Color(doc["r"], doc["g"], doc["b"]));
        strip.show();
    }
    else if (strcmp(type, "rainbow") == 0) {
        for (long hue = 0; hue < 5 * 65536; hue += 256) {
            for (int i = 0; i < strip.numPixels(); i++)
                strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(
                    hue + (i * 65536L / strip.numPixels()))));
            strip.show(); delay(5);
        }
    }
}
```

```

        websocket.loop(); // keep alive during animation!
    }
}
else if (strcmp(type, "off") == 0) {
    strip.clear(); strip.show();
}
else if (strcmp(type, "brightness") == 0) {
    strip.setBrightness(doc["value"]);
    strip.show();
}
}

// — WebSocket Event —————
void websocketEvent(WStype_t type, uint8_t* payload, size_t len) {
    if (type == WStype_CONNECTED) {
        isConnected = true;
        // Send join message
        StaticJsonDocument<128> doc;
        doc["type"] = "join";
        doc["device"] = "LED_ESP32";
        String msg; serializeJson(doc, msg);
        websocket.sendTXT(msg);
    }
    else if (type == WStype_DISCONNECTED) { isConnected = false; }
    else if (type == WStype_TEXT) { handleMessage((char*)payload); }
}

// — setup() —————
void setup() {
    Serial.begin(115200);

    strip.begin();
    strip.setBrightness(50);
    strip.show();

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nWiFi OK: " + WiFi.localIP().toString());

    websocket.begin(SERVER_HOST, SERVER_PORT, "/");
    websocket.onEvent(websocketEvent);
    websocket.setReconnectInterval(5000);
}

// — loop() —————
void loop() {
    websocket.loop(); // ALWAYS FIRST in loop!

    // Optional: auto-reconnect WiFi
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.begin(ssid, password);
        delay(3000);
    }
}

```

6. Complete Combined Template — WebSocket + FastLED

```

#include <WiFi.h>
#include <WebSocketsClient.h>
#include <ArduinoJson.h>
#include <FastLED.h>

```

```
#define LED_PIN    5
#define NUM_LEDS  30
CRGB leds[NUM_LEDS];
WebSocketsClient webSocket;
bool isConnected = false;

const char* ssid      = "YOUR_WIFI";
const char* password   = "YOUR_PASS";
const char* SERVER_HOST = "192.168.1.100";
const int   SERVER_PORT = 8080;

void handleMessage(char* payload) {
    StaticJsonDocument<512> doc;
    if (deserializeJson(doc, payload)) return;
    const char* type = doc["type"];

    if (strcmp(type, "color") == 0) {
        fill_solid(leds, NUM_LEDS, CRGB(doc["r"], doc["g"], doc["b"]));
        FastLED.show();
    }
    else if (strcmp(type, "rainbow") == 0) {
        fill_rainbow(leds, NUM_LEDS, 0, 7);
        FastLED.show();
    }
    else if (strcmp(type, "off") == 0) {
        FastLED.clear(); FastLED.show();
    }
    else if (strcmp(type, "brightness") == 0) {
        FastLED.setBrightness(doc["value"]);
        FastLED.show();
    }
}

void websocketEvent(WStype_t type, uint8_t* payload, size_t len) {
    if (type == WStype_CONNECTED) { isConnected = true; }
    else if (type == WStype_DISCONNECTED) { isConnected = false; }
    else if (type == WStype_TEXT) { handleMessage((char*)payload); }
}

void setup() {
    Serial.begin(115200);
    FastLED.addLeds<WS2812B, LED_PIN, GRB>(leds, NUM_LEDS);
    FastLED.setBrightness(50);
    FastLED.clear(); FastLED.show();

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); }

    webSocket.begin(SERVER_HOST, SERVER_PORT, "/");
    webSocket.onEvent(websocketEvent);
    webSocket.setReconnectInterval(5000);
}

void loop() {
    webSocket.loop(); // Always first!
}
```

7. Things You Might Have Missed

7.1 Non-Blocking Animations with millis()

Using `delay()` inside `loop()` blocks `webSocket.loop()` and will drop your connection. Use `millis()` instead:

```

unsigned long lastLedUpdate = 0;
const unsigned long LED_INTERVAL = 20; // ms between frames
static uint8_t hue = 0;

void loop() {
  websocket.loop(); // never blocked by delay()

  unsigned long now = millis();
  if (now - lastLedUpdate >= LED_INTERVAL) {
    lastLedUpdate = now;
    fill_rainbow(leds, NUM_LEDS, hue++, 7);
    FastLED.show();
  }
}

```

7.2 ESP32 Analog Read & ADC Quirks

If your exam involves reading sensors alongside LEDs:

```

analogReadResolution(12); // Set to 12-bit (0-4095), default on ESP32
// analogReadResolution(10); // 10-bit (0-1023) like Arduino Uno

int val = analogRead(34); // Read GPIO 34 (ADC1 only – avoid ADC2 with WiFi!)

// IMPORTANT: ADC2 (GPIO 0,2,4,12,13,14,15,25,26,27) is shared with WiFi
// When WiFi is active, ADC2 may give wrong readings!
// Use ADC1 pins: GPIO 32-39 for reliable analog reads with WiFi.

```

 **ADC2 pins (GPIO 0,2,4,12–15,25,26,27) do NOT work reliably when WiFi is active. Use ADC1 pins (GPIO 32–39) for analog input while using WebSocket.**

7.3 Serial Debug — Your Best Friend

```

void setup() {
  Serial.begin(115200); // Must match Serial Monitor baud rate!
  Serial.println("=== Starting ===");
}

// Print variable values:
Serial.print("R="); Serial.print(r);
Serial.print(" G="); Serial.print(g);
Serial.print(" B="); Serial.println(b); // println adds newline

// Print formatted:
Serial.printf("Connected! IP: %s\n", WiFi.localIP().toString().c_str());
Serial.printf("[LED] Pixel %d set to R=%d G=%d B=%d\n", i, r, g, b);

```

7.4 StaticJsonDocument Size — How to Pick

```

// Use ArduinoJson Assistant at https://arduinojson.org/v6/assistant/
// Rule of thumb:

StaticJsonDocument<64> doc; // 1-2 fields, small values
StaticJsonDocument<128> doc; // 3-5 simple fields
StaticJsonDocument<256> doc; // Typical outgoing message
StaticJsonDocument<512> doc; // Typical incoming message
StaticJsonDocument<1024> doc; // Complex nested / arrays (Tank game style)

```

```
// If you get garbage or empty values, your doc is too small!
// DynamicJsonDocument is an alternative (heap-allocated, more flexible):
// DynamicJsonDocument doc(512);
```

⚠ If your JSON has more data than StaticJsonDocument can hold, deserialization silently truncates or fails. When in doubt, use a LARGER buffer (1024 is safe for most exam tasks).

7.5 NeoPixel: ColorHSV (Built-In!)

```
// NeoPixel has its own HSV support (added in library v1.6.0+):
// ColorHSV(hue, sat, val)
// hue: 0-65535, sat: 0-255, val: 0-255

// Rainbow loop using ColorHSV:
for (long firstHue = 0; firstHue < 5*65536; firstHue += 256) {
  for (int i = 0; i < strip.numPixels(); i++) {
    int pixelHue = firstHue + (i * 65536L / strip.numPixels());
    // gamma32 applies perceptual brightness correction
    strip.setPixelColor(i, strip.gamma32(strip.ColorHSV(pixelHue)));
  }
  strip.show();
  delay(5);
}
```

7.6 WebSocket TLS / Secure Connection (wss://)

```
// Standard (ws://) - use begin()
websocket.begin("192.168.1.100", 8080, "/");

// Secure (wss://) - use beginSSL() or beginSSLWithCA()
websocket.beginSSL("example.com", 443, "/");
// OR to skip certificate verification (easier but insecure):
websocket.beginSSLWithCA("example.com", 443, "/", nullptr, true);
// (the last "true" = do not verify server cert)

// The Tank game uses port 443 (wss) - check if your exam server does too!
```

7.7 Button Debouncing

```
// Simple debounce for a push button (active LOW with INPUT_PULLUP)
#define BTN_PIN 27

unsigned long lastDebounce = 0;
const int DEBOUNCE_MS = 50;
bool btnState = false;
bool prevBtnState = false;

void setup() {
  pinMode(BTN_PIN, INPUT_PULLUP);
}

void checkButton() {
  bool reading = !digitalRead(BTN_PIN); // LOW when pressed
  if (reading != btnState) {
    if (millis() - lastDebounce > DEBOUNCE_MS) {
      lastDebounce = millis();
      btnState = reading;
    }
  }
}
```

```

    }
    if (btnState && !prevBtnState) {
        // BUTTON JUST PRESSED (rising edge)
        sendFire(); // or trigger LED effect
    }
    prevBtnState = btnState;
}

```

7.8 Troubleshooting Quick Reference

Symptom	Likely Cause	Fix
LEDs flicker or wrong color	Bad GND / power	Verify common GND; add 1000µF cap on 5V/GND near strip
First LED wrong color	No series resistor	Add 300-500Ω resistor on data line
LEDs all white on start	No strip.show() at init	Call strip.clear() + strip.show() in setup()
WebSocket disconnects constantly	delay() in loop()	Replace delay() with millis()-based timing
JSON values are 0 or null	Doc buffer too small	Increase StaticJsonDocument<N> size
strcmp always returns non-zero	Using == on const char*	Use strcmp(a,b) == 0 to compare strings
ADC reads garbage with WiFi on	Using ADC2 pin	Switch to ADC1 pins (GPIO 32-39)
No Serial output	Wrong baud rate	Check Serial Monitor is set to 115200
Won't upload to ESP32	Wrong board selected	Tools → Board → ESP32 Dev Module; hold BOOT button while uploading if needed
Colors look wrong (RGB vs GRB)	Wrong color order	Change NEO_GRB↔NEO_RGB or GRB↔RGB in library config

7.9 NeoPixel vs FastLED — Quick Comparison

Feature	NeoPixel	FastLED
Color type	uint32_t via Color(r,g,b)	CRGB struct, direct r/g/b fields
HSV support	strip.ColorHSV(h,s,v)	CHSV(h,s,v) — simpler
Fill all	strip.fill(color)	fill_solid(leds, N, color)
Rainbow	Manual with ColorHSV loop	fill_rainbow(leds, N, hue, delta)
Gradient	Manual	fill_gradient_RGB() built-in
Brightness	setBrightness() — lossy	setBrightness() — non-destructive
Animation tools	Fewer built-in helpers	More built-in effects & math
Ease of use	Simpler for beginners	More powerful for effects

8. Exam Day Checklist

Before You Start Coding

1. Connect ESP32 via USB. Open Arduino IDE. Set Board → ESP32 Dev Module, Port → your COM port.
2. Verify libraries: Adafruit NeoPixel, FastLED, WebSocketsClient, ArduinoJson — all installed.
3. Wire strip: Data → GPIO 5, Strip GND → PSU GND + ESP32 GND, Strip 5V → PSU 5V.
4. Open Serial Monitor at 115200 baud before uploading.
5. Test LEDs with a simple fill + show BEFORE adding WebSocket code.

Configuration to Update in Every Sketch

- ssid and password — your WiFi credentials
- SERVER_HOST and SERVER_PORT — exam server IP and port
- LED_PIN / LED_COUNT (NeoPixel) or NUM_LEDS (FastLED) — match your physical strip
- TANK_NAME or device identifier if the server expects one

The 5 Functions You Must Know Cold

Function	Library	Purpose
strip.setPixelColor(i, Color(r,g,b))	NeoPixel	Set one pixel color
strip.show()	NeoPixel	Push colors to strip — NOTHING WORKS WITHOUT THIS
leds[i] = CRGB(r,g,b); FastLED.show()	FastLED	Set pixel and push
websocket.loop()	WS	Keep WebSocket alive — must be in every loop()
deserializeJson(doc, payload)	ArduinoJson	Parse incoming JSON



Final check before upload: Did you call show()? Is websocket.loop() the FIRST line of loop()? Is your StaticJsonDocument large enough? Is your LED_COUNT correct?